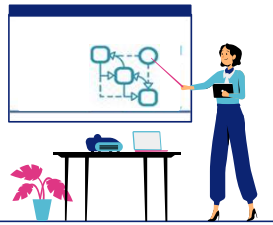
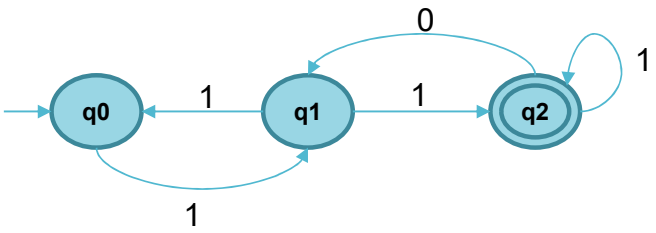
Chapter 3

Deterministe and non-deterministe Automata



Definitions

Activity

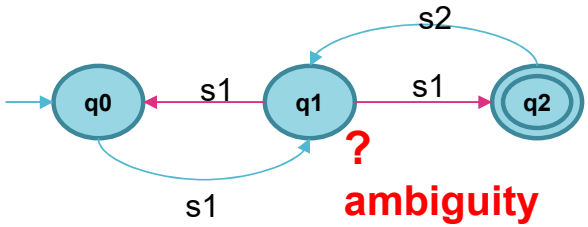


- ➔ Give the transition function corresponding to this automaton
- ➔ Is the string 101 belongs to this language , Is it modeled?
- ➔ Is the string 1111 belongs to this Language ? , how can you obtain it.
- ➔ Is the string 1101 belongs to this language ?

Definitions

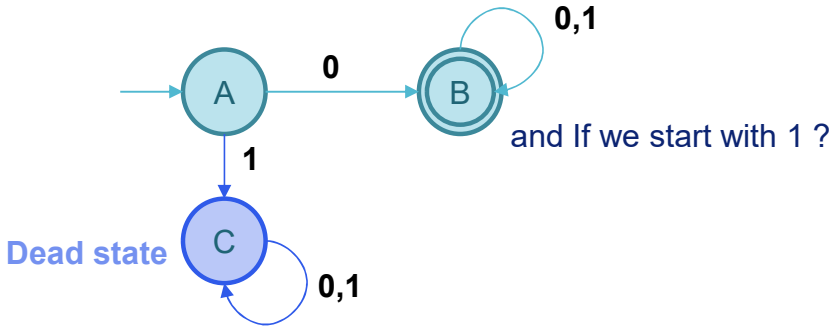
Meaning of nondeterminism

- Several choices may exists for the next choice at any point
- It cannot be deduced the next state until it triggers another transition.



Deterministic Finite Automata

- L=set of all strings that starts with '0' = {0,00,01,000,011,001,0000….}



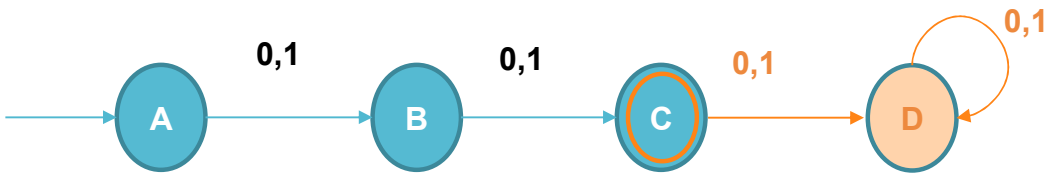
Now the Automata is deterministic (Deterministic Finite Automata – DFA)

All possibilities of acceptance and Rejection are modeled)

Deterministic Finite Automata

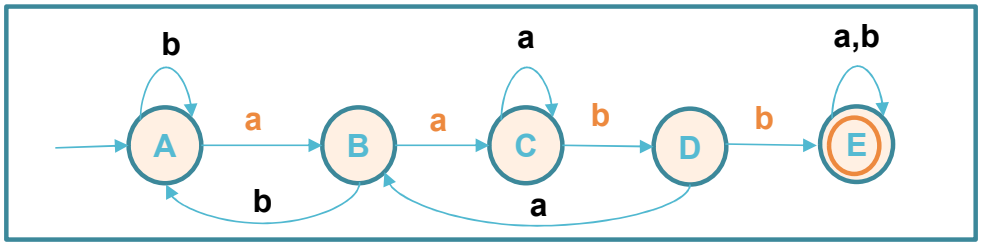
- Construct a DFA that accepts sets of all strings over {0,1} of length 2.

Σ ={0,1}
L={00,01,10,11}



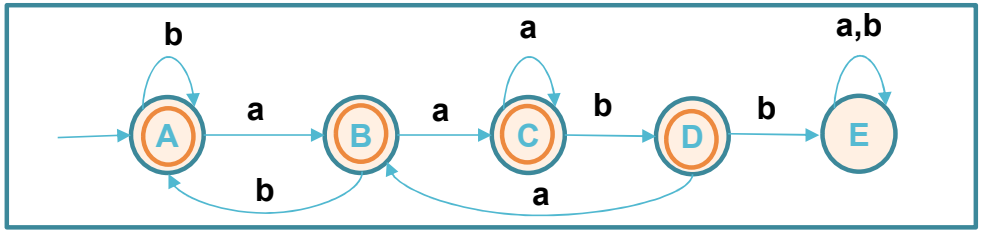
Deterministic Finite Automata

- Construct a DFA that accepts any strings over {a,b} that **does not** contain the string **aabb** in **Σ ={a,b}**
  - Construct a DFA that accepts over {a,b} that contains the string **aabb**



Flip the states

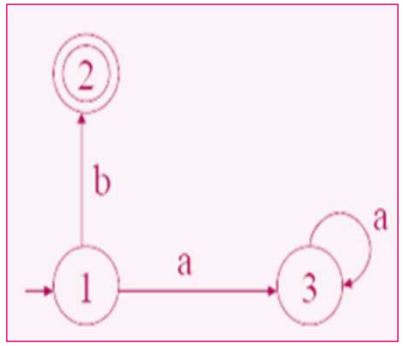
1. Make the final state into non final state
2. Make the non final states into final states



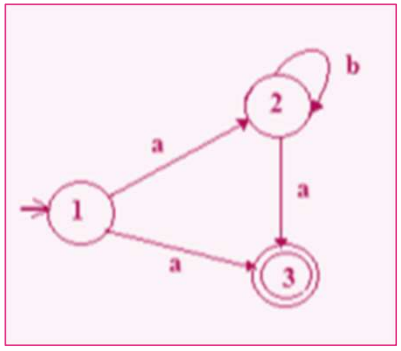
Definitions

- An automaton with an alphabet Σ is said to be **completely defined** if, for each symbol **s** ∈ Σ and each state **q**, there is at least one transition labeled **s** that leaves **q.**

- An automaton with an alphabet Σ is said to be **unambiguous** if, for each state **q** and each symbol **s** ∈ Σ, there exists **at most one** transition leaving **q** and labeled **s**..



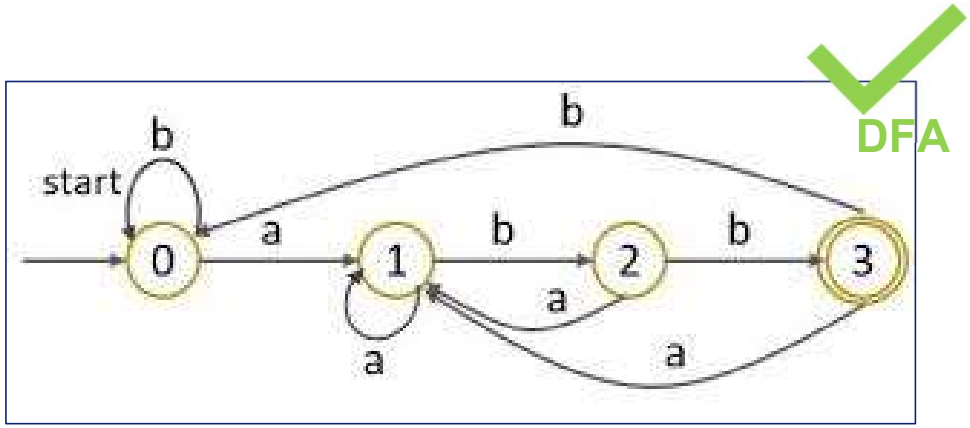
Not completely defined



Ambiguous

Deterministic Finite Automata

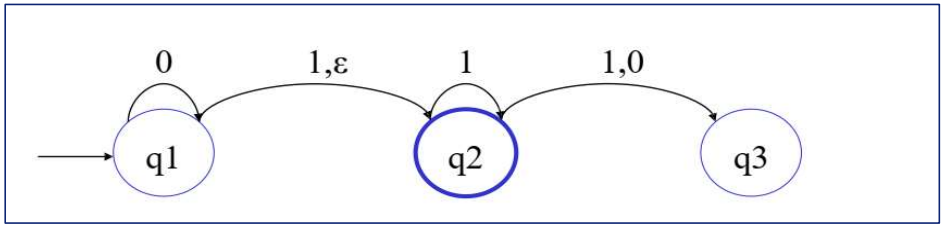
- Un automaton is called determinist if it is **completely defined** et does not present any **ambiguity**

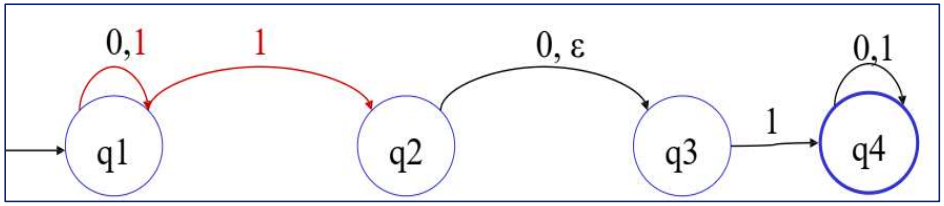


DFA vs NFA

- Un automaton is called **Non-deterministic (NFA)** if it presents **ambiguity :** NFA may have more than one transition labeled with a symbol

Examples





From NFA to DFA

- Let, **M = (Q, ∑, δ, q0, F)** is an **NFA** which accepts the language L(M).
- There should be equivalent **DFA** denoted by **M' = (Q', ∑', q0', δ', F')** such that L(M) = L(M').

Steps for converting NFA to DFA

Step 1: Initially Q' = ϕ

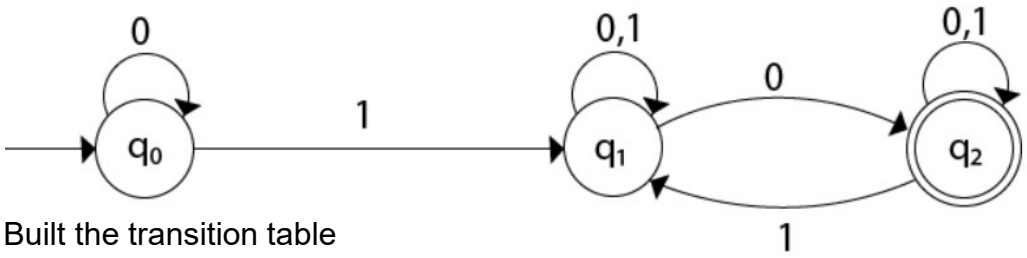
Step 2: Add q0 of NFA to Q'. Then find the transitions from this start state.

Step 3: In Q', find the possible set of states for each input symbol.
If this set of states is not in **Q'**, then add it to **Q'.**

Step 4: In DFA, the final state will be all the states which contain F(final states of NFA)

From NFA to DFA

Convert the given **NFA to DFA.**



Built the transition table

	0	**1**
→**q0**	q0	q1
q1	{q1, q2}	q1
q2	q2	{q1, q2}

From NFA to DFA

Example1

	0	1
→**q0**	q0	q1
q1	{q1, q2}	q1
q2	q2	{q1, q2}

	0	1
→**[q0]**	[q0]	[q1]
[q1]	[q1, q2]	[q1]
***[q2]**	[q2]	[q1, q2]
***[q1, q2]**	[q1, q2]	[q1, q2]

1.δ'([q0], 0) = **[q0]**
2.δ'([q0], 1) = **[q1]**

3.δ'([q1], 0) = **[q1, q2]** (➔ **new state**)
4.δ'([q1], 1) = **[q1]**

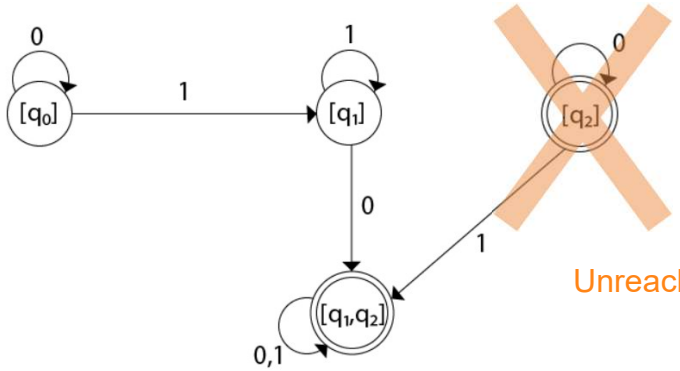
5.δ'([q2], 0) = **[q2]**
6.δ'([q2], 1) = **[q1, q2]**

7.δ'([q1, q2], 0) = δ(q1, 0) ∪ δ(q2, 0) = {q1, q2} ∪ {q2} = **[q1, q2]**
8. δ'([q1, q2], 1) = δ(q1, 1) ∪ δ(q2, 1) = {q1} ∪ {q1, q2} = {q1, q2} = **[q1, q2]**

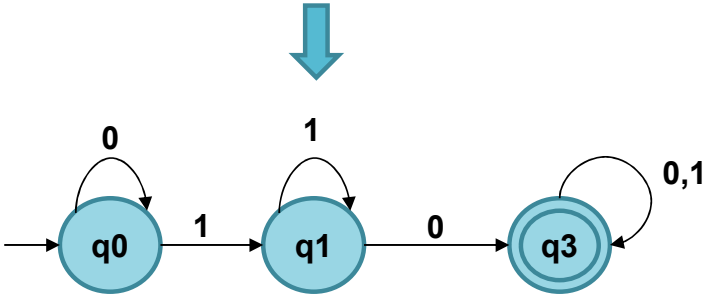
From NFA to DFA

Example1

	0	1
→**[q0]**	[q0]	[q1]
[q1]	[q1, q2]	[q1]
***[q2]**	[q2]	[q1, q2]
***[q1, q2]**	[q1, q2]	[q1, q2]

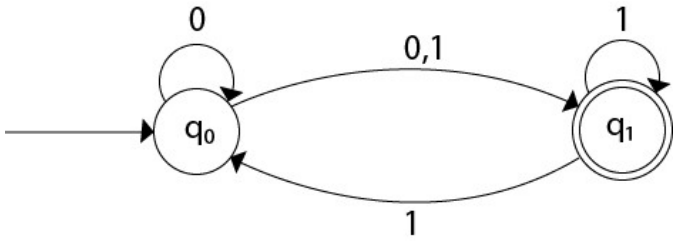


Unreachable State



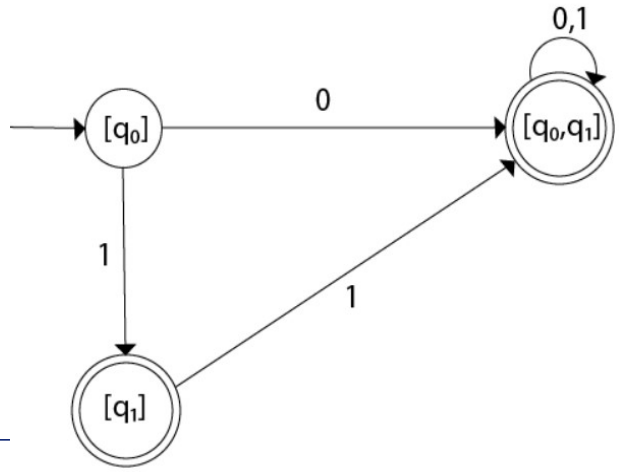
From NFA to DFA

Example2 :



	0	1
→q0	{q0, q1}	{q1}
q1	ϕ	{q0, q1}

δ'([q0], **0**) = **[q0,q1]** ➔ new state
δ'([q0], **1**) = **[q1]**
δ'([q1], **0**) = ϕ
δ'([q1], **1**) = **[q0,q1]**
δ'([q0,q1], **0**) = **[q0,q1]**
δ'([q0,q1], **1**) = **[q0,q1]**



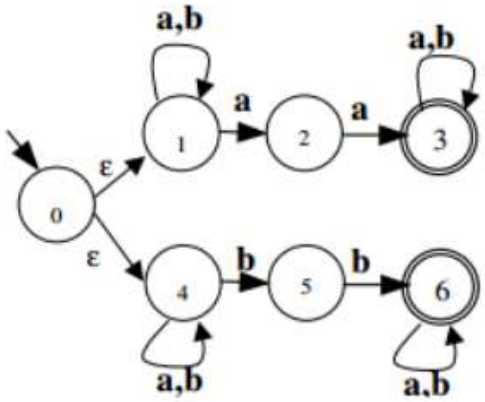
78

che - Bachelor GLSI2 2023 PRISTINI SCHOOL OF AI

Automata

Application

Create the corresponding DFA to this automaton (NFA→ DFA)



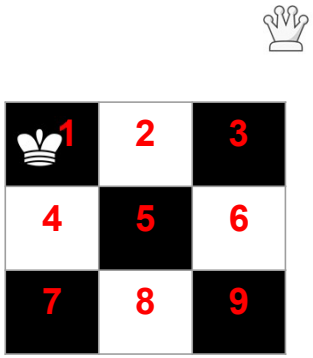
From NFA to DFA

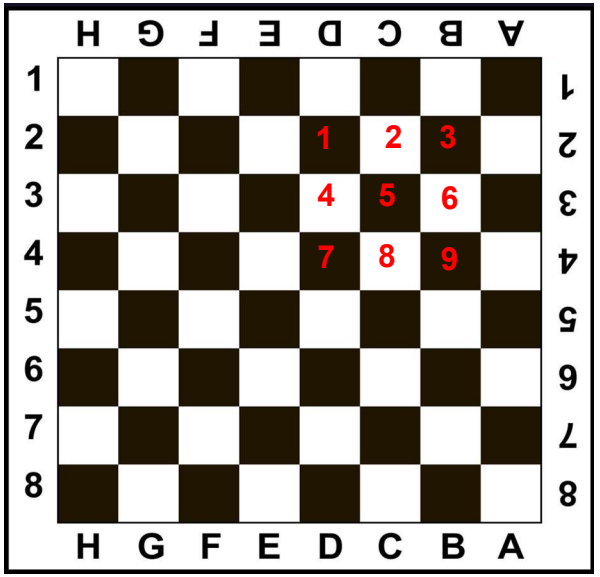
Example : Moves on Chessboard

States = squares.

- Inputs =**W**(move to an adjacent white square) and **B** (move to an adjacent black square).
- Start state **F2** , final state are in opposite corners **D4**

	W	B
1	2,4	5
2	4,6	1,3,5
3	2,6	5
4	2,8	1,5,7
5	2,4,6,8	1,3,7,9
6	2,8	3,5,9
7	4,8	5
8	4,6	5,7,9
9	6,8	5



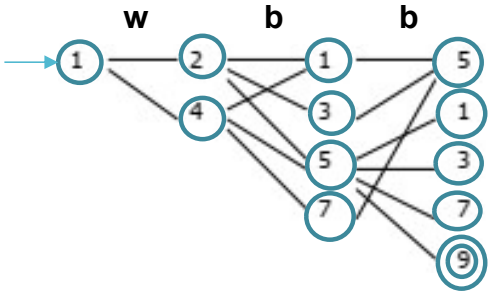


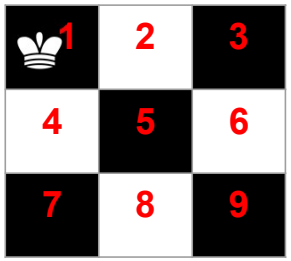
From NFA to DFA

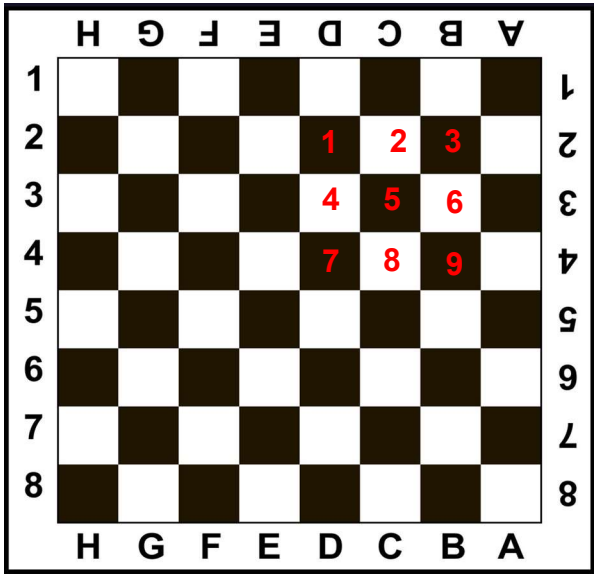
Example : Moves on Chessboard

States = squares.

- Inputs = **w**(move to an adjacent white square) and **b** (move to an adjacent black square).
- Start state **F2** , final state are in opposite corners **D4**



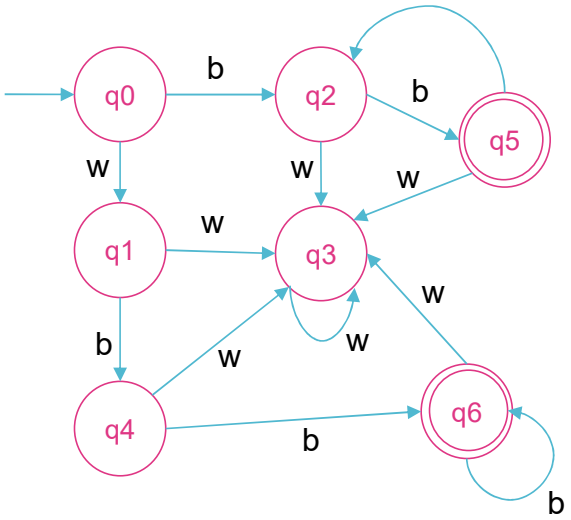




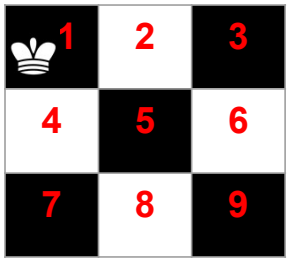
From NFA to DFA

Example : Moves on Chessboard

All moves on chessboard

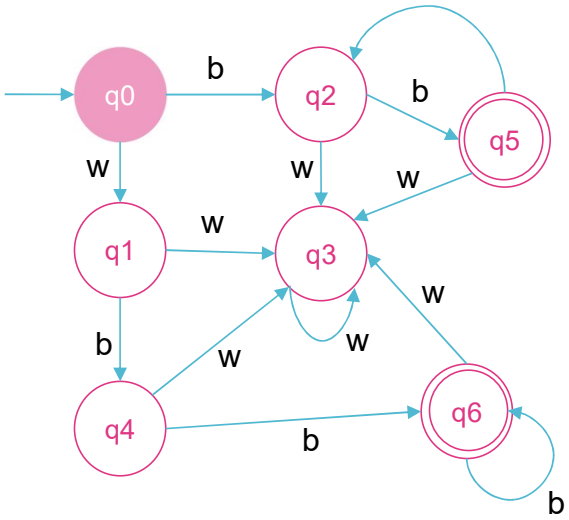


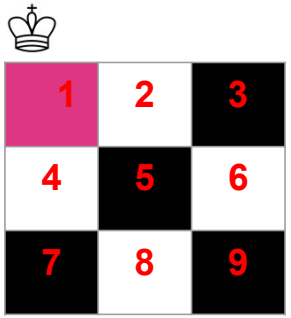
		w	b
q0	→ {1}	{2,4}	{5}
q1	{2,4}	{2,4,6,8}	{1,3,5,7}
q2	{5}	{2,4,6,8}	{1,3,7,9}
q3	{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
q4	{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
q5	* {1,3,7,9}	{2,4,6,8}	{5}
q6	* {1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}



From NFA to DFA

Example : Moves on Chessboard

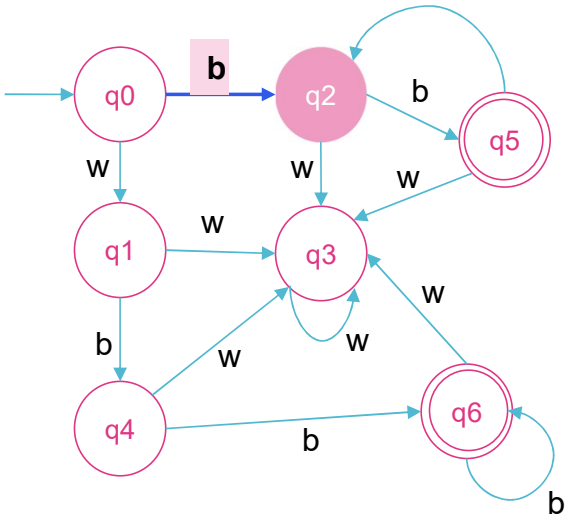


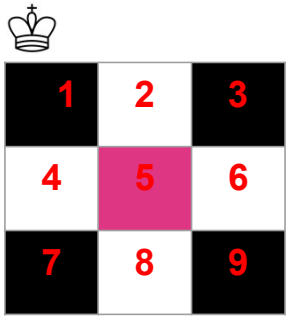


	w	b
→ q0 {1}	{2,4}	{5}
q1 {2,4}	{2,4,6,8}	{1,3,5,7}
q2 {5}	{2,4,6,8}	{1,3,7,9}
q3 {2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
q4 {1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
* q5 {1,3,7,9}	{2,4,6,8}	{5}
* q6 {1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

From NFA to DFA

Example : Moves on Chessboard

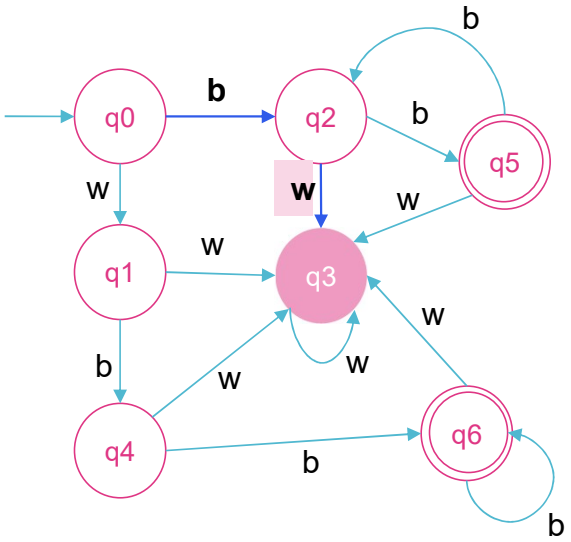


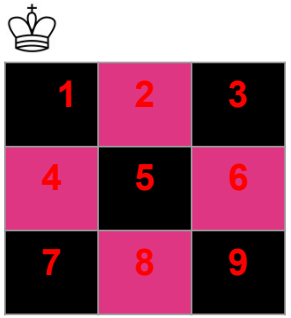


		w	b
q0	→ {1}	{2,4}	{5}
q1	{2,4}	{2,4,6,8}	{1,3,5,7}
q2	{5}	{2,4,6,8}	{1,3,7,9}
q3	{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
q4	{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
q5	* {1,3,7,9}	{2,4,6,8}	{5}
q6	* {1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

From NFA to DFA

Example : Moves on Chessboard

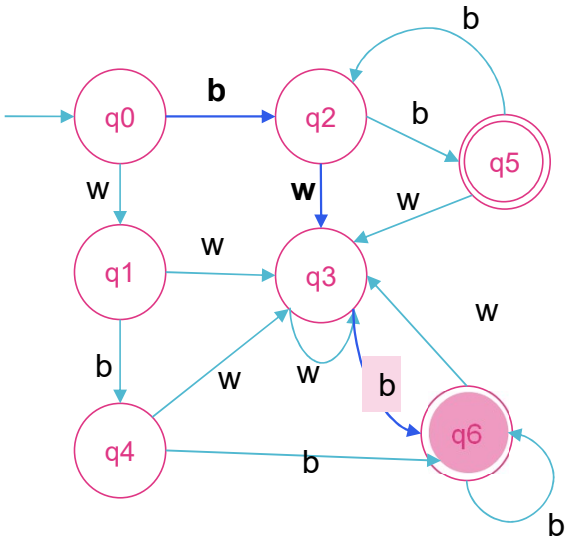


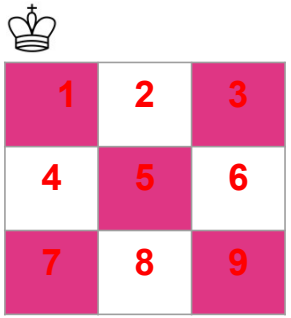


		w	b
q0	→ {1}	{2,4}	{5}
q1	{2,4}	{2,4,6,8}	{1,3,5,7}
q2	{5}	{2,4,6,8}	{1,3,7,9}
q3	{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
q4	{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
q5 *	{1,3,7,9}	{2,4,6,8}	{5}
q6 *	{1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

From NFA to DFA

Example : Moves on Chessboard





		w	b
q0	→ {1}	{2,4}	{5}
q1	{2,4}	{2,4,6,8}	{1,3,5,7}
q2	{5}	{2,4,6,8}	{1,3,7,9}
q3	{2,4,6,8}	{2,4,6,8}	{1,3,5,7,9}
q4	{1,3,5,7}	{2,4,6,8}	{1,3,5,7,9}
q5 *	{1,3,7,9}	{2,4,6,8}	{5}
q6 *	{1,3,5,7,9}	{2,4,6,8}	{1,3,5,7,9}

Automata

Software tools

Regular expression to DFA : regexper.com

Automata simulator : https://automatonsimulator.com/